

TRABAJO DE FIN DE GRADO

Orchestration Trade-offs in Edge Computing: Systemd vs Kubernetes

Análisis de rendimiento, distribución de cargas e inferencia de modelos de IA en el extremo de la red

Autor:

[Tu Nombre y Apellidos]

Director/es:

[Nombre del Director/a]

Grado en Ingeniería [Informática / Telecomunicaciones]

[Nombre de tu Universidad]

[Mes y Año de la convocatoria]

Índice

Índice de figuras

Índice de cuadros

Capítulo 1

Introducción

1.1 Contexto y Motivación

El auge del *Edge Computing* plantea nuevos retos en la orquestación de servicios descentralizados. Los entornos perimetrales poseen recursos limitados que cuestionan la viabilidad de modelos *cloud-native* puros.

1.2 Objetivos del Trabajo

El objetivo principal es analizar y comparar los *trade-offs* (rendimiento, consumo y complejidad) entre una orquestación basada en Systemd (nativa y ligera) frente a Kubernetes (robusta pero pesada) en escenarios Edge.

1.3 Estructura de la Memoria

El documento se estructura siguiendo la progresión del desarrollo técnico iterativo, desde la base teórica hasta la comparativa final.

Capítulo 2

Estado del Arte y Tecnologías Involucradas

Este capítulo compila la investigación técnica previa (correspondiente a las *Setmanas 4 y 5*) necesaria para fundamentar el diseño arquitectónico del proyecto.

2.1 Paradigmas de Computación: Cloud vs. Edge Computing

Definición de conceptos y necesidades de orquestación en el extremo.

2.2 Tecnologías de Contenedorización Ligera

Análisis de Podman frente a Docker, destacando su arquitectura *daemonless* y ejecución *rootless*, ideales para entornos perimetrales seguros y con pocos recursos.

2.3 Orquestación y Ciclo de Vida en el Edge

2.3.1 Systemd y la evolución hacia Quadlets

Gestión del ciclo de vida de los contenedores de forma nativa desde el sistema operativo.

2.3.2 Distribuciones Ligeras de Kubernetes (K3s)

Concepto de K3s como estándar de facto para llevar la semántica de Kubernetes al Edge.

2.4 Protocolos de Comunicación y Serialización para Alta Frecuencia

2.4.1 Arquitecturas HTTP/REST y RPC (gRPC)

2.4.2 Mensajería Asíncrona y Punto a Punto (ZeroMQ, MQTT)

2.4.3 Estrategias de Serialización: JSON vs Protobuf vs MessagePack

Comparativa teórica entre texto plano con alto *overhead* y esquemas binarios eficientes.

2.5 Frameworks de Machine Learning para Inferencia

Introducción a PyTorch y las características del dataset MNIST utilizado para las cargas de trabajo.

Capítulo 3

Metodología y Evolución de la Arquitectura

3.1 Enfoque de Desarrollo Iterativo

El proyecto se ha abordado mediante iteraciones incrementales semanales. Se partió del aprovisionamiento base (*Setmana 0 a 3*), pasando por la optimización de protocolos (*Setmana 4*), la integración de IA (*Setmana 5*) y culminando en la migración a Kubernetes (*Setmana 6*).

3.2 Definición del Entorno de Pruebas

Infraestructura compuesta por un nodo Master (Ubuntu 22.04, WSL) y nodos Workers (Ubuntu 24.04, VMware VMs).

3.3 Herramientas de Aprovisionamiento y Automatización

Justificación del uso de Ansible para garantizar despliegues reproducibles bajo el patrón de *Side-loading* (evitando repositorios de contenedores externos).

3.4 Casos de Uso y Definición de Tareas

3.4.1 Tarea 1: Transmisión masiva de datos (Conteo de imágenes)

División del dataset en N particiones y retorno del conteo.

3.4.2 Tarea 2: Inferencia distribuida de Inteligencia Artificial

Sustitución del conteo simple por la predicción numérica usando modelos pre-entrenados.

Capítulo 4

Fase 1: Arquitectura Base y Optimización de Red (Systemd + Podman)

4.1 Despliegue de la Infraestructura mediante Systemd Quadlets

Explicación de cómo se estructuraron los servicios base de forma agnóstica a la carga de trabajo.

4.2 Diseño del Banco de Pruebas de Comunicaciones

Metodología para medir latencias y consumos enviando 60.000 imágenes a los nodos.

4.3 Resultados: Comparativa de Rendimiento de Protocolos

Análisis del rendimiento base (HTTP/1.1), el estándar microservicios (gRPC), el crudo P2P (ZeroMQ) y el paradigma IoT (MQTT).

4.4 Resultados: Impacto de la Serialización

La transición de JSON a binario (Protobuf) y el refinamiento final con MessagePack (*schema-less*).

4.5 Selección de la Arquitectura Óptima para el Edge

Conclusión de esta fase: La elección de ZeroMQ + MessagePack como pila de red ganadora para integrar en Systemd.

Capítulo 5

Fase 2: Implementación del Flujo de Machine Learning

5.1 Entrenamiento y Empaquetado del Modelo (PyTorch)

El nodo Master asume la carga computacional de entrenamiento y serializa el modelo (`.pth`).

5.2 Distribución Segura de Modelos y Datos a Nodos Trabajadores

Envío del modelo y las particiones mediante la arquitectura de red optimizada en la Fase 1.

5.3 Ejecución de Inferencia Descentralizada

Los Workers cargan el modelo en memoria y evalúan cada imagen, devolviendo las predicciones agregadas al Master.

5.4 Análisis de Rendimiento y Consumo Computacional

Evaluación del incremento de estrés en CPU y memoria (T_{proc}) al añadir el *overhead* matemático de los tensores.

Capítulo 6

Fase 3: Migración a Kubernetes y Análisis Comparativo Final

6.1 Transición de Arquitectura: De Quadlets a Manifestos de K3s

Adaptación de los servicios a la semántica de *Deployments* y *Services* de K8s.

6.2 Despliegue del Escenario de Inferencia sobre K3s

Ejecución de la Tarea 2 a través de la interfaz de red de contenedores (CNI) de Kubernetes.

6.3 Comparativa de Trade-offs: Systemd vs. Kubernetes

6.3.1 Plano de Gestión: Consumo de RAM y CPU

Comparativa del *overhead* en reposo del Kubelet frente al demonio de Systemd.

6.3.2 Plano de Datos: Latencia de red y Throughput

Impacto del enrutamiento de Kube-proxy frente a la red nativa punto a punto.

6.3.3 Complejidad Operativa y Facilidad de Mantenimiento

Capítulo 7

Conclusiones y Trabajo Futuro

7.1 Conclusiones de la Investigación

Resumen de los hitos alcanzados y la viabilidad técnica demostrada a lo largo de los capítulos.

7.2 Respuesta al Dilema: ¿Systemd o Kubernetes para el Edge?

Veredicto final apoyado en las métricas extraídas en la Fase 3. Cuándo merece la pena asumir el coste de Kubernetes y cuándo es preferible la simplicidad de Systemd.

7.3 Líneas de Trabajo Futuro

Bibliografía

- [1] Red Hat, *Podman Documentation*, [Online]. Disponible en: <https://podman.io/docs/>
- [2] Freedesktop.org, *systemd.service — Service unit configuration*, [Online].
- [3] Cloud Native Computing Foundation (CNCF), *gRPC: A high performance, open source universal RPC framework*, [Online].
- [4] P. Hintjens, *ZeroMQ: Messaging for Many Applications*, O'Reilly Media, 2013.
- [5] Rancher Labs (SUSE), *K3s Lightweight Kubernetes*, [Online]. Disponible en: <https://k3s.io/>
- [6] Paszke, A. et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS 2019.